

■ ****ExportController.java Explanation (Tanglish)****

****Package Declaration****

package com.inventory.controller;

➔■ Ithu file oda package name, controller package la iruku nu indicate pannum.

****Imports****

➔■ Product, ProductService, ExcelExportService – necessary classes import panniruka.

➔■ Spring annotations & classes (Autowired, RestController, etc.) use panniruka.

➔■ Java built-in imports (List, IOException, ByteArrayInputStream).

****Class Declaration****

@RestController

@RequestMapping("/api/export")

➔■ Ithu REST API controller nu solludhu.

➔■ API path start: /api/export/...

****Dependency Injection****

@Autowired private ProductService productService;

@Autowired private ExcelExportService excelExportService;

➔■ Spring automatically object create pannum (autowiring).

****Excel Export Endpoint****

@GetMapping("/products/excel")

public ResponseEntity exportProductsToExcel() throws IOException

➔■ API endpoint to download Excel file.

****Steps Inside the Method:****

1■■■ `productService.getAllProducts()` → Fetch all product data from DB.

2■■■ `excelExportService.exportProductsToExcel(products)` → Convert data to Excel format.

3■■■ `HttpHeaders headers = new HttpHeaders();` → Header for download name.

4■■■ `headers.add("Content-Disposition", "attachment; filename=products.xlsx");`

→ Browser la file name products.xlsx nu set aagum.

5■■■ Return ResponseEntity with:

- Status: OK (200)

- Headers: Download info

- ContentType: application/vnd.ms-excel

- Body: Excel data stream

****Flow Summary:****

1■■■ User calls `/api/export/products/excel`

2■■■ Backend fetches all products

3■■■ Converts to Excel file

4■■■ Sends Excel file to browser

5■■■ Browser starts downloading "products.xlsx" ■